

Diagonal Difference

30 more points to get your first star!

Rank: 6388785 | Points: 0/30

Problem

Submissions

Leaderboard

Discussions

Editorial

Given a square matrix, calculate the absolute difference between the sums of its diagonals.

For example, the square matrix `arr` is shown below:

```
1 2 3
4 5 6
9 8 9
```

- The left-to-right diagonal = $1 + 5 + 9 = 15$.
- The right-to-left diagonal = $3 + 5 + 9 = 17$.

Their absolute difference is $|15 - 17| = 2$.

Function description

Complete the `diagonalDifference` function with the following parameter:

- `int arr[n][m]`: a 2-D array of integers

Return

- `int`: the absolute difference in sums along the diagonals

Input Format

The first line contains a single integer, `n`, the number of rows and columns in the square matrix `arr`.

Each of the next `n` lines describes a row, `arr[i]`, and consists of `n` space-separated integers `arr[i][j]`.

Constraints

- $-100 \leq arr[i][j] \leq 100$

Author [vatsalchanana](#)Difficulty [Easy](#)Max Score [10](#)Submitted By [1867564](#)

NEED HELP?

[View discussions](#)[View editorial](#)[View top submissions](#)

RATE THIS CHALLENGE



MORE DETAILS

[Download problem statement](#)[Download sample test cases](#)[Suggest Edits](#)

```
1 import java.util.*;
2
3 class Result {
4
5     public static int diagonalDifference(List<List<Integer>> arr) {
6         int n = arr.size();
7
8         int leftDiagonal = 0;
9         int rightDiagonal = 0;
10
11         for (int i = 0; i < n; i++) {
12             // Left-to-right diagonal
13             leftDiagonal += arr.get(i).get(i);
14
15             // Right-to-left diagonal
16             rightDiagonal += arr.get(i).get(n - 1 - i);
17         }
18
19         return Math.abs(leftDiagonal - rightDiagonal);
20     }
21 }
22
23 public class Solution {
24     public static void main(String[] args) {
25         Scanner sc = new Scanner(System.in);
26
27         int n = sc.nextInt();
28
29         List<List<Integer>> arr = new ArrayList<>();
30
31         for (int i = 0; i < n; i++) {
32             List<Integer> row = new ArrayList<>();
33
34             for (int j = 0; j < n; j++) {
35                 row.add(sc.nextInt());
36             }
37
38             arr.add(row);
39         }
40     }
41 }
```

Upload Code as File

☐ Test against custom input

Run Code

Submit Code

Congratulations!

You have passed the sample test cases. Click the submit button to run your code against all the test cases.

✓ Sample Test case 0

Input (stdin)

Download

```
1 3
2 11 2 4
3 4 5 6
4 10 8 -12
```

Your Output (stdout)

```
1 15
```

Expected Output

Download

```
1 15
```

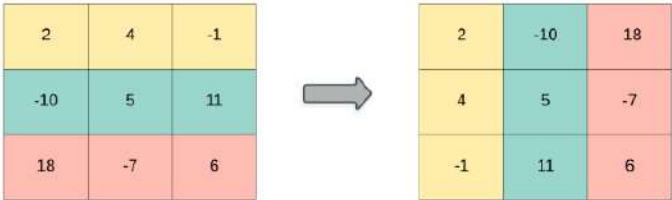
Description Editorial Solutions Submissions

867. Transpose Matrix

Easy Topics Companies Hint

Given a 2D integer array `matrix`, return the **transpose** of `matrix`.

The **transpose** of a matrix is the matrix flipped over its main diagonal, switching the matrix's row and column indices.



Example 1:

Input: `matrix = [[1,2,3],[4,5,6],[7,8,9]]`
Output: `[[1,4,7],[2,5,8],[3,6,9]]`

Example 2:

Input: `matrix = [[1,2,3],[4,5,6]]`
Output: `[[1,4],[2,5],[3,6]]`

Constraints:

- `m == matrix.length`
- `n == matrix[i].length`
- `1 <= m, n <= 1000`
- `1 <= m * n <= 105`

👍 4.3K 🗨️ 133 ☆ 📄 ⓘ

15 Online

</> Code

Java 🔒 Auto

≡ 📄 ↺ ↻ ↶ ↷

```
1 class Solution {
2     public int[][] transpose(int[][] matrix) {
3         int m = matrix.length;
4         int n = matrix[0].length;
5
6         int[][] result = new int[n][m];
7
8         for (int i = 0; i < m; i++) {
9             for (int j = 0; j < n; j++) {
10                 result[j][i] = matrix[i][j];
11             }
12         }
13
14         return result;
15     }
16 }
```

Saved

Ln 17, Col 1

☑️ Testcase > Test Result

Accepted Runtime: 0 ms

☑️ Case 1 ☑️ Case 2

Input

matrix =
[[1,2,3],[4,5,6],[7,8,9]]

Output

[[1,4,7],[2,5,8],[3,6,9]]

Expected

[[1,4,7],[2,5,8],[3,6,9]]

🤍 Contribute a testcase